

دليل إضافي

دليل إضافي: المزامنة والاستعادة

الهاتف أساس · SyncService · Supabase · الاستعادة برمز الطالب · الحذف من كل مكان

الشاشات: شاشة الاستعادة · ورقة الحذف

يسبقها: المرحلتان 2 و4

المشروع: رِزمة · تطبيق تعلّم يعمل دون إنترنت

وقت القراءة: 55 دقيقة

تتبعها: دليل تنزيل الدروس

المنصة: Flutter · Dart · Drift · Riverpod · go_router

1. الفكرة في سطرين

الفكرة

الهاتف هو الأساس.

الطالب يتعلّم ويلعب وكل شيء يُحفظ على هاتفه حتى دون إنترنت.

الخادم نسخة احتياطية فقط.

حين يتوفّر الإنترنت يرفع الهاتف نتائج الطالب إلى الخادم، فإن ضاع الهاتف أو تغيّر، يستعيد الطالب كل شيء برمزه.

تشبيه: الهاتف هو **دفتر الطالب**، والخادم **خزنة المدرسة** فيها نسخة مصوّرة من الدفتر. الطالب يكتب في الدفتر دائمًا، وتُحدَّث النسخة في الخزنة كلما مرّ على المدرسة (توفّر الإنترنت).

ملاحظة

هذا الدليل يغطي ميزة بُنيت

بعد

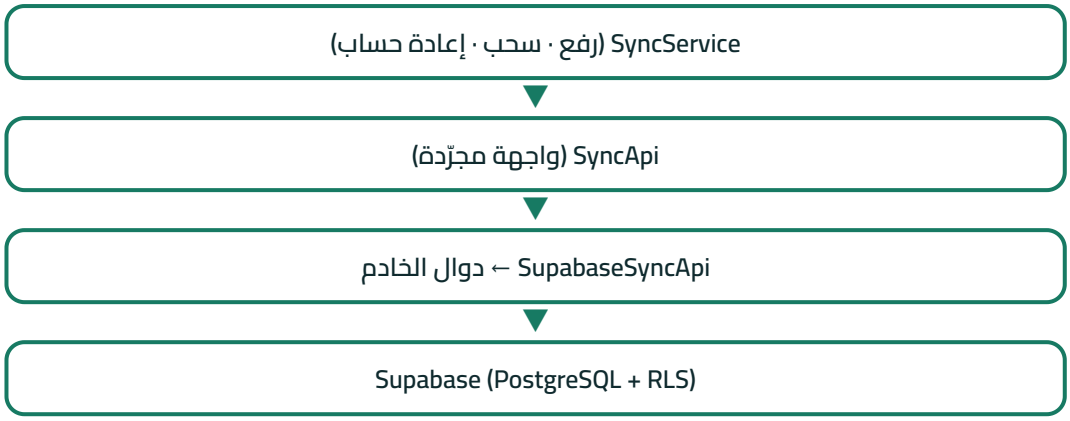
الخطة الأصلية للمراحل (ولذلك هو دليل إضافي). وهي مرتبطة بالمرحلتين 2 (الملفات ورمز الطالب) و4 (رمز الطالب والحذف).

ما الذي بُني؟ ستة أجزاء

#	الجزء	بكلمة	أين
1	قاعدة بيانات الهاتف (الإصدار 3 فما فوق)	جهّزنا الدفتر ليُصوَّر بلا أخطاء	`lib/core/db/`
2	الخادم (Supabase)	بنينا الخزنة وقوانينها	`supabase/`
3	عامل المزامنة	يأخذ الدفتر إلى الخزنة ويعود بالجديد	`sync/sync_service.dart`
4	شاشة الاستعادة	طالب بهاتف جديد يستعيد دفتره برمزه	`sync/restore_screen.dart`
5	أزرار الحذف في «حسابي»	إزالة من الهاتف فقط أو حذف من كل مكان	`account/widgets/delete_profile_sheet.dart`
6	الوصل بالخادم الحقيقي	فتح الخزنة وإعطاء الدفتر مفتاحها	`sync/supabase_sync_api.dart`

المزامنة **شغّالة** على خادم حقيقي. وللطالب لا شيء جديد: بدون إنترنت يعمل التطبيق كاملًا، ومع الإنترنت تُحفظ نسخة من نتائجه في الخلفية.

الهاتف (SQLite) ← مصدر الحقيقة



2. ماذا يُرفع وماذا يبقى على الهاتف؟ (الخصوصية)

يُرفع إلى الخادم	يبقى على الهاتف فقط
معرّف الطالب، الصف، رمز الطالب، رمز الصف، لون التميمة، الملابس، الإشارات المرجعية، النتائج (المحاولات)، المشتريات، أجزاء الدروس المنجزة	**الاسم**، **الرقم السري**، إعدادات الهاتف (حجم الخط...)، الإتقان والتقدّم (يُحسبان)

> **انتبه**
>
> **الخادم لا يعرف من الطالب:**
>
> لا اسم ولا رقم هاتف ولا بريد ولا رقم سري. لذلك تطلب الاستعادة
>
> **الاسم من جديد**
>
> ، ولا يُستعاد PIN أبدًا (هو قفل بين الإخوة على الهاتف نفسه فقط).

نوعان من البيانات

النوع	أمثلة	القاعدة
أحداث لا تتغيّر	المحاولات، المشتريات، إنجاز جزء درس	إذا حصلت مرة فقد انتهى الأمر. الخادم يحفظها مرة واحدة، وإن وصلت مرتين يتجاهل الثانية.
أشياء تتغيّر	ملف الطالب (الصف، اللون، الملابس)، الإشارات المرجعية	يعدّلها الطالب، فلها قواعد خاصة (العدّادات أدناه).

الإتقان والتقدّم يُحسبان ولا يُرفعان

الإتقان وتقدّم الدروس **ناتج** من المحاولات والأجزاء المنجزة، فلا يُرفعان. دالة `recomputeProgress` (المرحلة 7ب) تعيد حسابهما **من الصفر** من الأحداث. نستعملها بعد الاستعادة، أو حين تصل محاولات من هاتف آخر. واختبار يتأكد أن نتيجتها **مطابقة بالضبط** لما يحسبه التطبيق أثناء اللعب.

3. تصميم قاعدة الهاتف للمزامنة

قبل أن نصوّر الدفتر يجب ترتيبه حتى لا يحدث خلط حين يستعمل الطالب **هاتفين**. هذه التعديلات، واحدةً واحدة:

أ. معرّف المحاولة صار UUID

كانت كل محاولة تأخذ رقمًا متسلسلًا 1، 2، 3. لو عمل أنس على هاتفه المحاولة رقم 5 وعملت ليان على هاتف آخر محاولتها رقم 5، فعند وصولهما للخادم: من هي رقم 5؟ **تعارض.** الحل: كل محاولة تأخذ **UUID** عشوائيًا طويلًا مثل `3b55b4cb-05ee-487d-9078-79940857ffab` (احتمال تطابق هاتفين صفر عمليًا).

ب. علامة «انرفع» (synced_at)

كل نتيجة وشراء وجزء درس فيه خانة «متى انرفع؟»: **فارغة** = ما زالت على الهاتف فقط، و**فيها وقت** = وصلت الخزنة بسلام. مثل علامة صح على الورقة بعد تصويرها.

ج. عدّاد التعديلات (local_version): سباق «عدّلتُ وأنا أرفع»

الهاتف يرفع الإشارات المرجعية وفي اللحظة نفسها يزيل الطالب درسًا منها. يرد الخادم «وصلني». لو علّمناها «انرفعت» لضاع التعديل الجديد! الحل: كل تعديل يزيد عدّادًا (1، 2، 3). الهاتف يرفع ويتذكّر «رفعتُ النسخة 2»، وعند الردّ يفحص: هل العدّاد ما زال 2؟ **نعم** ← علّمها «انرفعت»، **لا** (صار 3) ← لا تعلّمها، تُرفع في المرة القادمة.

د. رقم نسخة الملف (revision): تعارض بين هاتفين

أنس عنده هاتفان بلا إنترنت. على الأول غيّر **الصف** إلى السادس، وعلى الثاني غيّر **اللون** إلى المرجاني. الخادم يعطي كل نسخة رقمًا. حين يرفع الهاتف الثاني يقول الخادم: «عندي نسخة أحدث» (تعارض). فيقوم الهاتف الثاني بما يلي:

1. يأخذ نسخة الخادم (فيها الصف السادس).
2. يعيد وضع **ما غيّره هو فقط** (اللون المرجاني).
3. يرفع مرة ثانية.

النتيجة: كلا الهاتفين عندهما الصف السادس **و** اللون المرجاني. لا تعديل ضاع.

هـ. الإزالة «علامة» لا حذف

لو أزال الطالب درسًا من إشاراته المرجعية على هاتف وحذفنا السطر، فالهاتف الثاني ما زال يحتفظ بالدرس فيعيده! لذلك الإزالة **لا تحذف السطر**، تكتب عليه «أُزيل بتاريخ كذا»، فيعرف الهاتف الثاني أنه أُزيل فعلًا.

و. رصيد النجوم لا يكون سالبًا أبدًا

عند ليان 30 نجمة. دون إنترنت اشترت شالًا بـ 30 على هاتف ونظارة بـ 30 على آخر. بعد المزامنة: 60 − 30 − 30 = −30. القرار: القطع **لا تُسحب** منها، والرصيد المعروض **صفر**، ولا تستطيع شراء جديد حتى تجمع نجومًا تغطي «الدَّين». (انظر `WardrobeRepository.balance` بـ `clamp` في المرحلة 8.)

ز. بصمة لكل سؤال

كل نتيجة تحفظ بصمة السؤال (16 حرفًا تتغيّر إذا تغيّر نصه) فتعرف التقارير أي نسخة من السؤال أجاب عنها الطالب.

ح. رمز الطالب أطول

`RZ-XXXX-XXXX` : ثمانية أحرف ≈ 853 مليار احتمال (مقابل ≈ 923 ألف للقديم ذي الأربعة)، لأنه صار **المفتاح الوحيد** لاستعادة الملف فلا بدّ أن يكون صعب التخمين. `normalize` يقبله بأي حالة وبمسافات أو بلا شرطات.

4. الخادم (Supabase)

الخادم PostgreSQL على Supabase، ملفاته في `supabase/` : `migrations/` (الجداول والدوال والصيانة) و`tests/database/` (اختبارات pgTAP) و`README.md` (العقد بين التطبيق والخادم).

رقم الدور (server_seq) والمؤشر

كل سطر يُكتب على الخادم يأخذ **رقم دور** متزايدًا (101، 102، 103...). يحفظ الهاتف «آخر رقم رأيتُه لهذا الطالب = 102». وفي المرة القادمة يسأل: «أعطني كل شيء بعد 102» وهذا هو **المؤشر** (cursor)؛ مثل علامة القراءة في كتاب.

قفل لكل طالب

لو رفع هاتفان للطالب نفسه في اللحظة نفسها قد ينتهي الرقم 102 قبل 101، فيقرأ الهاتف حتى 102 ويتأخر 101 فلا يسحبه أحد أبدًا! الحل: قبل أي كتابة **يقفل الخادم الطالب** فيكتب هاتف واحد في كل مرة وتخرج الأرقام **بالترتيب دائمًا**.

الدوال التي يناديها الهاتف

لا أحد يكتب في الجداول مباشرة؛ الهاتف يستدعي دوالّ محددة فقط:

الدالة	ما تفعله
`upload_batch`	رفع دفعة (حدّها 200 سطر). المكرر يُتجاهل. تعديل الملف يُقبل أو يعيد «تعارض».
`pull_changes`	سحب الجديد بعد مؤشر الطالب، صفحةً صفحة.
`restore_profile`	الاستعادة برمز الطالب: يربط هذا الهاتف بالطالب.
`request_deletion`	الحذف النهائي من كل مكان (برمز الطالب).
`unlink_device`	فكّ ربط هذا الهاتف فقط.

الحماية

- **RLS:** كل هاتف يرى طلابه فقط.
- **منع التخمين:** 5 محاولات خاطئة على **الرمز نفسه** ← يُقفل 15 دقيقة. و10 محاولات فاشلة من **الهاتف نفسه** خلال ساعة ← يُقفل الهاتف ساعة.
- **لا قفل على عنوان الإنترنت**: لأن مدرسة كاملة قد تكون على شبكة واحدة، فقفل العنوان يقفلها كلها.

الصيانة اليومية (تعمل وحدها 3:17 فجرًا)

ماذا	متى يُمسح
ملف طُلب حذفه نهائيًا	بعد 14 يومًا (وخلالها يستطيع إلغاء الحذف برمزه)
ملف بلا أي نشاط	بعد 24 شهرًا (لا 12، بسبب النزوح)
«هويات» مجهولة لم ترتبط بطالب	بعد 30 يومًا

> **انتبه**
>
> المفتاح داخل التطبيق `sb_publishable_...` **مفتاح عام** مصمَّم ليكون في التطبيق (مثل عنوان المدرسة: الكل يعرفه ولا يفتح الخزنة). ما يحمي الطلاب هو **قواعد الخزنة** (RLS والدوال). ويمكن تغييره عند البناء بـ `--dart-define=RIZMA_SUPABASE_URL` و `RIZMA_SUPABASE_KEY`، أو إيقاف المزامنة كلها بـ `RIZMA_SYNC=off`.

5. عامل المزامنة (SyncService)

هو الذي «يمشي» بين الهاتف والخادم. يعمل **وحده في الخلفية**: عند فتح التطبيق، وعند عودة الطالب إليه، وكل 10 دقائق. **الطالب لا يراه أبدًا.**

الواجهة المجرّدة SyncApi

lib/features/sync/sync_api.dart — سطر 6

```
abstract interface class SyncApi {
  Future<void> ensureSignedIn();

  Future<Map<String, dynamic>> uploadBatch(Map<String, dynamic> batch);

  Future<Map<String, dynamic>> pullChanges(
    String profileId,
    int cursor, {
    int limit = 500,
  });

  Future<Map<String, dynamic>> restoreProfile(
    String code, {
    bool cancelDeletion = false,
  });

  Future<Map<String, dynamic>> requestDeletion(String profileId, String code);

  Future<void> unlinkDevice(String profileId);
}

const maxBatchRows = 200;
```

لماذا؟

الخدمة تتكلم مع `SyncApi` فقط. في التطبيق الحقيقي `SupabaseSyncApi`، وفي الاختبارات `FakeSyncServer` يطبّق نفس قوانين الخادم الحقيقي. فتُختبر كل الحالات الصعبة بلا إنترنت.

خادم وهمي

كل مزامنة ثلاث خطوات

lib/features/sync/sync_service.dart — سطر 53

```
Future<SyncOutcome> _syncProfile(
  String profileId, {
  required bool retried,
}) async {
  try {
    await _api.ensureSignedIn();
    await _uploadAll(profileId);
    if (await _pullAll(profileId)) await _recompute(profileId);
    return SyncOutcome.done;
  } on SyncApiException catch (e) {
    switch (e.error) {
      case SyncError.network:
        return SyncOutcome.offline;
      case SyncError.notLinked:
        return SyncOutcome.needsRestore;
      case SyncError.profileDeleted:
        await _removeLocal(profileId);
        await _forget(profileId);
        return SyncOutcome.removed;
      case SyncError.profileUnknown when !retried:
        await _markAllUnsynced(profileId);
        return _syncProfile(profileId, retried: true);
      default:
        return SyncOutcome.failed;
    }
  }
}
```

1. **ارفع** كل ما خانة «انرفع» فيه فارغة، على **دفعات** (200 سطر أو 256 كيلوبايت).
2. **اسحب** كل جديد بعد مؤشر الطالب (مثل محاولات عملها على هاتف آخر).
3. **أعد حساب** الإتقان والتقدّم إن وصل شيء جديد (`_recompute`).

لماذا دفعات؟

طالب غاب أسبوعين دون إنترنت قد يملك مئات النتائج. طلب واحد كبير مع إنترنت ضعيف يسهل أن ينقطع ويعود من الأول كل مرة. مع الدفعات: لو انقطع بعد أول 200 **فالـ 200 وصلت وعُلّمت** والباقي يكمل لاحقًا، ولا شيء يتكرر.

الحالات الخاصة (SyncOutcome)

النتيجة	متى	ما يفعله
`done`	نجحت	ينتهي

النتيجة	متى	ما يفعله
`offline`	لا إنترنت أو انقطاع	يتوقف بهدوء ويحاول لاحقًا، ولا شيء يضيع
`needsRestore`	الهاتف غير مربوط بالطالب	يطلب الاستعادة برمز الطالب
`removed`	طُلب الحذف من هاتف آخر	يزيل الملف من هذا الهاتف
`failed`	خطأ آخر	يعيد المحاولة لاحقًا
profileUnknown	الخادم نسي الطالب (مُسح لعدم النشاط)	يعلّم كل شيء «غير مرفوع» ويعيد الرفع كله

الجدولة في الخلفية

lib/features/sync/sync_providers.dart سطر 32

```
class SyncScheduler extends ConsumerStatefulWidget {
  const SyncScheduler({
    super.key,
    required this.child,
    this.interval = const Duration(minutes: 10),
  });

  final Widget child;
  final Duration interval;

  @override
  ConsumerState<SyncScheduler> createState() => _SyncSchedulerState();
}

class _SyncSchedulerState extends ConsumerState<SyncScheduler>
    with WidgetsBindingObserver {
  Timer? _timer;
  bool _running = false;

  @override
  void initState() {
    super.initState();
    if (ref.read(syncServiceProvider) == null) return;
    WidgetsBinding.instance.addObserver(this);
    _timer = Timer.periodic(widget.interval, (_) => _sync());
    WidgetsBinding.instance.addPostFrameCallback((_) => _sync());
  }

  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    if (state == AppLifecycleState.resumed) _sync();
  }

  Future<void> _sync() async {
    final service = ref.read(syncServiceProvider);
    if (service == null || _running) return;
    _running = true;
    try {
      await service.syncAll();
    } catch (_) {
```

- مؤقّت كل **10 دقائق** + مزامنة عند أول إطار + عند `AppLifecycleState.resumed`.
- العلَم `_running` يمنع مزامنتين متداخلتين.
- لو `syncServiceProvider` يعيد `null` (المزامنة مطفأة) لا يعمل شيء.
- أي خطأ يُبتلع (`catch (_) {}`) لأن المزامنة **لا يجوز أن تُظهر خطأ للطالب**.

6. الاتصال بالخادم الحقيقي

الهوية المجهولة

كل هاتف يدخل الخادم **بهوية مجهولة** (بلا اسم ولا بريد) ويربط بها طلابه. لهذه الهوية «تذكرة» تتجدّد كل فترة، و**التذكرة القديمة تُلغى** عند التجديد. لذلك:

- يحفظ الهاتف التذكرة الجديدة **فورًا** بعد كل تجديد (`SessionStore` في جدول app_settings). لو لم يحفظها فإنه سيدخل في المرة التالية بهوية جديدة لا تعرف طلابه.
- لو انقطع الإنترنت **في منتصف التجديد** فلا ينشئ هوية جديدة؛ ينتظر ويجرّب لاحقًا.

lib/features/sync/supabase_sync_api.dart — سطر 16

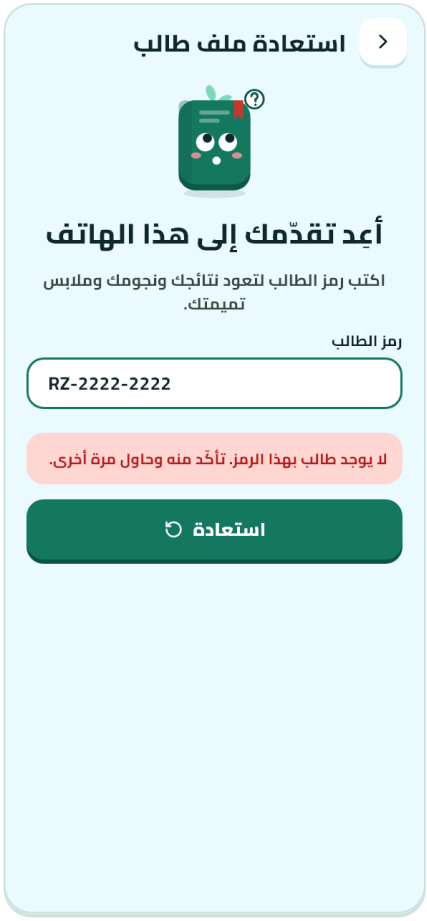
```
  static ServerConfig? fromEnvironment() {
    if (const String.fromEnvironment('RIZMA_SYNC') == 'off') return null;
    return const ServerConfig(
      url: String.fromEnvironment(
        'RIZMA_SUPABASE_URL',
        defaultValue: 'https://znrxseznkjmcugmyzmya.supabase.co',
      ),
      publishableKey: String.fromEnvironment(
        'RIZMA_SUPABASE_KEY',
        defaultValue: 'sb_publishable_lULv0NY-Urx00TyuQoYBEQ_9S5MHHvo',
      ),
    );
  }
}

abstract interface class SessionStore {
```

عندما تسوء الشبكة

الحالة	ما يحدث
لا إنترنت، أو الطلب أطول من 30 ثانية (90 للرفع)	«انقطاع»: يُعاد في المرة القادمة ولا شيء يضيع
الخادم مشغول أو متوقف	نفس الشيء: انقطاع
انتهت التذكرة أو رُفضت	يدخل من جديد

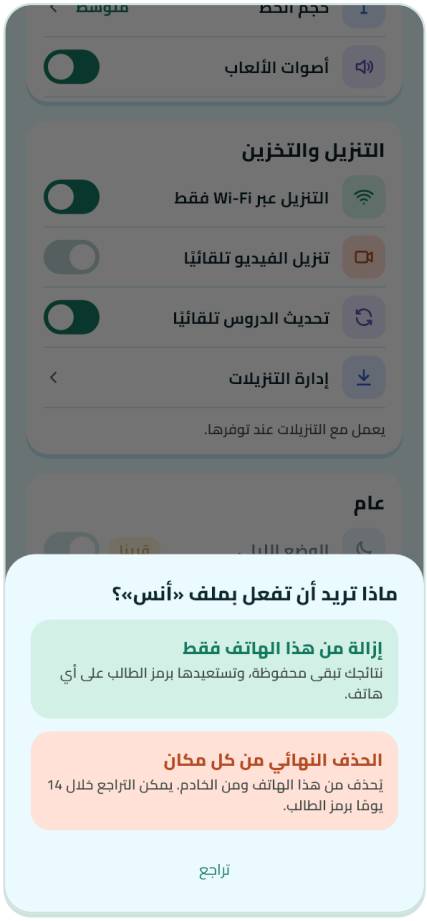
7. شاشة الاستعادة وحذف الملف



رمز طالب خاطئ



وجدنا ملفك: الاسم



حذف الملف: الخياران

الاستعادة (RestoreScreen)

متى؟ ضاع هاتف الطالب أو اشترى هاتفًا جديدًا. **من أين؟** زر «لديّ ملف طالب على هذا الجهاز» في الترحيب، أو «لديك رمز طالب؟» في شاشة اختيار الملف (`AppRoutes.restore`).

1. يكتب الطالب **رمز الطالب** (بأي شكل: كبير/صغير، بشرطات أو بلا).
2. الخادم يتأكد ويربط الهاتف الجديد بالطالب (`restore_profile`).
3. يقول التطبيق «وجدنا ملفك!» ويطلب **الاسم** من جديد (لأنه لا يُرفع).
4. يسحب كل النتائج ويعيد حساب التقدّم ويدخل الطالب إلى دروسه.

`lib/features/sync/sync_service.dart` سطر 11

```
enum RestoreStatus {
  restored,
  wrongCode,
  codeLocked,
  deviceLocked,
  pendingDeletion,
  offline,
}
```

لكل حالة رسالة واضحة: رمز خاطئ · رمز مقفل (حاول بعد كذا دقيقة) · هاتف مقفل · لا إنترنت · الملف قيد الحذف (مع زر «إلغاء الحذف واستعادة الملف»).

حذف الملف مع خادم

بدون خادم (المزامنة مطفأة): كما في المرحلة 4: PIN ← تأكيد ← حذف نهائي من الهاتف. **مع خادم**: بعد PIN خياران:

الخيار	ما يحدث
إزالة من هذا الهاتف فقط	يحاول **رفع** النتائج أولًا (`syncProfile`). إن رُفع كل شيء: «نتائجك محفوظة» ويعرض رمز الطالب. إن بقيت نتائج غير مرفوعة (لا إنترنت): **تحذير أحمر** أنها ستضيع، وزر «تراجع» هو الكبير. النسخة على الخادم **تبقى** فيستعيدها الطالب برمزه.
الحذف النهائي من كل مكان	يجب كتابة **رمز الطالب**. يختفي الملف فورًا، ويُمسح من الخادم بعد 14 يومًا، وخلالها يستطيع **إلغاء الحذف** من شاشة الاستعادة.



تحذير: نتائج غير مرفوعة

lib/features/account/widgets/delete_profile_sheet.dart سطر 156

```
Future<void> _remove() async {
  setState(() => _busy = true);
  Haptics.heavy();
  try {
    await _service!.unlink(widget.profile.id);
  } catch (_) {}
  if (mounted) _close(DeleteOutcome.removedFromPhone);
}
```

8. الاختبارات وكيف نثق أن كل شيء يعمل

الملف	ما يغطيه
`test/features/schema_v3_test.dart`	مخطط القاعدة للمزامنة (UUID، synced_at، العدّادات)
`test/features/sync_test.dart`	هاتفان عبر خادم وهمي: رفع، سحب، تعارض الصف واللون، انقطاع بنصف الرفع، تعديل أثناء الرفع، الإتقان والتقدّم المطابقان بعد الاستعادة
`test/features/restore_screen_test.dart`	شاشة الاستعادة بحالاتها
`test/features/account_delete_sync_test.dart`	الحذف مع الخادم بخياريه
`test/features/supabase_sync_api_test.dart`	الاتصال الحقيقي (مع HTTP وهمي): الهوية، الأخطاء، المهلات
`test/core/db_migration_test.dart`	ترقية القاعدة تحفظ البيانات
`supabase/tests/database/*.sql`	اختبارات الخادم (pgTAP) داخل PostgreSQL: القفل، منع التخمين، الحذف، الصيانة، الحماية

اختبار الاختبارات

للتأكد أن الاختبارات **تكشف الغلط فعلًا**، خُرّب الكود عمدًا مرتين: غُيّر قفل الرمز من 5 إلى 6 محاولات فسقط اختبار الخادم ✅، وأُزيل فحص عدّاد التعديلات فسقط اختبار السباق ✅؛ ثم أُعيد الكود الصحيح.

اختبار الحمل على الخادم

هاتفان لنفس الطالب رفعا **800 نتيجة في اللحظة نفسها** وهاتف ثالث يسحب طوال الوقت: لم تضع نتيجة، ولم تُحسب واحدة مرتين، ولم يفُت الهاتف الثالث شيء. وهذا بالضبط ما يحميه «قفل الطالب». السكربت في `tool/live/` (لا تشغّله على الخادم الحقيقي إلا بتنسيق مع الفريق).

> **انتبه**
>
> **لا تشغّل**
>
> `tool/live/*` من جهازك دون إذن: يكتب على خادم حقيقي ويترك طلابًا تجريبيين.

جرّبها بنفسك (بخادم وهمي)

1. شغّل `flutter test test/features/sync_test.dart` واقرأ اختبار «تعارض الصف واللون».
2. افتح `test/helpers/fake_sync_server.dart` وانظر كيف يطبّق قوانين الخادم (قفل، رقم دور، تعارض).
3. في التطبيق: «حسابي ← رمز الطالب» ولاحظ حالة «لم يُحفظ على الإنترنت بعد».

9. تدرّب وتحقّق من فهمك

قرارات تصميم وأسبابها

القرار	السبب
الهاتف أساس والخادم نسخة	التطبيق يعمل دون إنترنت (غزة).
UUID للمحاولات	لا تعارض بين هاتفين.
عدّاد local_version	لا يضيع تعديل أثناء الرفع.
revision والدمج	لا يضيع تعديل بين هاتفين.
الإزالة علامة لا حذف	الهاتف الثاني يعرف أنها أُزيلت.
الاسم لا يُرفع	الخادم لا يعرف من الطالب (خصوصية).
دفعات 200 سطر	انقطاع الإنترنت لا يُعيد العمل من الصفر.

أسئلة

س1. لماذا تطلب الاستعادة الاسم من جديد؟

س2. ما الذي يضمنه «قفل الطالب» في الخادم؟ وما الذي كان يحدث بدونه؟

س3. عدّل الطالب إشاراته المرجعية أثناء رفعها. كيف نضمن ألا يضيع التعديل؟

س4. انقطع الإنترنت بعد رفع أول 200 من 500 نتيجة. ماذا يحدث؟

س5. لماذا لا نقفل المحاولات الفاشلة على عنوان الإنترنت؟

تمارين

1. اقرأ `supabase/README.md` واكتب بكلماتك العقد بين التطبيق والخادم (ست نقاط).
2. اكتب اختبارًا للخادم الوهمي: رمز خاطئ ست مرات ← هل يُقفل؟
3. ارسم مخطّطًا لتدفق الاستعادة كاملًا (من الرمز إلى دخول الدروس).

الإجابات

ج1. لأن الاسم لا يُرفع إلى الخادم أبدًا (خصوصية)، فالخادم لا يملكه ليعيده.

ج2. يضمن أن هاتفًا واحدًا يكتب للطالب في كل مرة، فتخرج أرقام server_seq بالترتيب. بدونه قد ينتهي رقم 102 قبل 101 فلا يسحب أحد الأخير أبدًا.

ج3. عدّاد local_version: يرفع الهاتف ويتذكّر النسخة، وعند الردّ لا يعلّمها «انرفعت» إلا إن بقي العدّاد كما هو؛ وإلا تُرفع في المرة القادمة.

ج4. الـ 200 الأولى وصلت وعُلّمت، ويكمل الباقي (300) في المزامنة القادمة دون تكرار.

ج5. لأن مدرسة كاملة قد تكون على عنوان واحد، فقفله يمنع المدرسة كلها.

مصطلحات

المصطلح	المعنى
Sync	تحديث النسخة على الخادم من الهاتف والعكس
UUID	رقم عشوائي طويل لا يتكرر
synced_at	«متى انرفع؟» (فارغ = لم يُرفع)
local_version	عدّاد التعديلات على الهاتف
revision	رقم نسخة الملف على الخادم
server_seq	رقم الدور لكل سطر على الخادم
cursor / مؤشر	آخر رقم دور رآه الهاتف
RLS	Row Level Security: كل هاتف يرى طلابه فقط
pgTAP	أداة اختبارات داخل PostgreSQL
Anonymous identity	هوية مجهولة للهاتف على الخادم

ما بعد هذا الدليل

تنزيل الدروس يُشرح في **دليل تنزيل الدروس**، والتبادل بين الهواتف بلا خادم في **دليل المشاركة بين الهواتف**.